Logan MacAskill

logan.macaskill.com | github.com/log2bits | (650) 237-9593
Los Altos, CA | linkedin.com/in/logan-macaskill-356190222 | logan@macaskill.com

Experience

Software Engineering Intern | **Flickr** | **Jun 2023 - Aug 2023**

- Cut compute cost **92%** and raised throughput **39%** on Wolverine, a production Python AWS Lambda photo-repair service running over a library of ~10 billion photos, by tuning memory and thread allocation to the cheapest stable configuration.
- Profiled dozens of Lambda configurations on controlled batches of real jobs in Splunk, modeling cost per job, throughput, and tail latency to find the cost/performance point AWS docs do not surface.
- Recovered AWS's undocumented vCPU-to-memory scaling ratio by measuring CPU behavior across memory settings and fitting a linear regression, enabling accurate per-job cost estimates.

Software Engineering Intern | **SmugMug** | **Jun 2022 - Aug 2022**

- Owned a user-facing gallery stats feature **end to end** in PHP, React, and Next.js, adding daily refresh scheduling and clearer labels and empty states, shipped through PR review and QA.
- Wrote unit tests and backend changes supporting a **release-critical** PHP 8.1 upgrade, pairing with senior engineers to land it on schedule.
- Traced and fixed a broken checkout-flow link buried in a large monorepo, shipping the fix through a multi-stage review and QA pipeline.

Projects

ray-vox: Ray-traced voxel renderer | **Rust, WebGPU, WGSL** | **2026 - Present**

- Designed a custom pointerless, bitpacked sparse-voxel tree (4^3 branching) for a from-scratch ray-traced renderer in Rust and WebGPU, storing an 84 MB, 22-million-voxel scene in **15.3 MB** (**5.5x smaller** than the source, and smaller than its max-compression gzip) while remaining directly GPU-traversable with no decode step.
- Eliminated all per-node child pointers using dual 64-bit occupancy bitmasks with popcount-implicit indexing, packing interior and leaf offsets into a single 32-bit word (13 + 19 bits, static-asserted), which cut memory while keeping GPU traversal branchless and warp-coherent.
- Wrote a WGSL fragment shader that ray-traces the full scene at real-time frame rates via a stack-based tree-DDA with per-cell empty-space skipping, using no rasterization, hardware ray tracing, or separate acceleration structure, and matched the on-disk and in-VRAM layouts so a world loads in **2.3 ms** as a near-verbatim copy.

Crowd Surfers: Real-time 3D game | **Godot, GLSL** | **2025 - 2026**

- Led a mid-project migration from a faked-depth sprite system to true 3D with an angled orthographic camera, building a working prototype that convinced a **100-person student team** to adopt the rewrite.
- Built a 3D occlusion-based transparency shader in GLSL that fades buildings as the player skates behind them, using a camera-to-player frustum test plus dithered alpha to fit the alpha-cut asset pipeline.
- Added real-time shadows, dynamic lighting, and camera feel (speed-based FOV, screen shake, look-ahead), all smoothed with interpolation.

Coaxial Swerve Drive | **Java, Control Theory, Computer Vision** | **2023 - 2024**

- Built a coaxial swerve drivetrain from scratch for FRC (custom CNC chassis, electronics, ~2,000 lines of Java) with no prior team experience, leading an off-season team to a working prototype in one month, **~4 months ahead of schedule**.
- Ran per-module PID and feedforward at a **1 kHz control loop** and derived current limits from robot mass and tire friction to maximize grip without slip or brownout, after self-teaching graduate-level control theory.
- Fused wheel encoders, gyro, and AprilTag vision through a Kalman-filter pose estimator for **sub-centimeter localization** robust to wheel slip, and simulated the full robot (torque curves, inertia, battery, brownouts) on the exact production code.

Game Server Infrastructure | **AWS, Linux, Docker** | **2020 - Present**

- Ran a self-managed, multi-service game server on AWS EC2 for years of live users, isolating each Minecraft and Factorio instance in its own Docker container via a Pterodactyl panel, behind a custom domain and DNS.
- Built a snapshot-based backup and migration workflow that re-hosts the whole stack in minutes, using it to scale instance size on demand and move the server to **AWS Graviton (ARM)** for lower cost at equal performance.
- Wrote a two-way Discord bridge (plugin plus bot) syncing in-game chat, roles, and permissions bidirectionally, and kept the stack healthy over SSH on Ubuntu through config edits, log triage, and plugin and version troubleshooting under load.

Skills

Languages: Rust, C++, C, Python, Java, TypeScript / JavaScript, PHP, Swift, SQL, Assembly, GLSL / WGSL
Data & ML: NumPy, Pandas, Matplotlib, scikit-learn, PyTorch, TensorFlow, Jupyter, MATLAB, R
Tools & Cloud: Git, Linux, AWS (EC2, Lambda, S3, Graviton), Docker, Nginx, Node.js, Splunk, OpenCV, Godot, React / Next.js
Systems & Graphics: Performance Optimization, Multithreading, SIMD, Profiling (perf, flamegraphs), CMake, GPU Data Structures, Vulkan, WebGPU, Ray Tracing, Real-Time Rendering, Shaders, Level-of-Detail
Concepts & Practices: Control Theory, Computer Vision, Pose Estimation, Agile / Scrum, Unit Testing, Code Review

Education

University of California, Santa Cruz | **Santa Cruz, CA** | **Expected Jun 2028**
B.S. in Computer Science | GPA: 3.6/4.0
Honors: Merit Scholarship, Dean's Honors